Containerization_and_DevOps

🐳 Docker & Docker Compose Lab README

📌 Overview

This lab demonstrates:

- Difference between **Docker Run (imperative)** and **Docker Compose (declarative)**
- Running **single-container** and **multi-container applications**
- Converting Docker commands into Compose files
- Using **Dockerfile with Compose**
- Advanced concepts like **multi-stage builds and resource limits**

🔷 PART A – Docker Run vs Docker Compose

Docker Run (Imperative)

```
docker run -d \
  --name lab-nginx \
  -p 8081:80 \
  -v $(pwd)/html:/usr/share/nginx/html \
  nginx:alpine
```

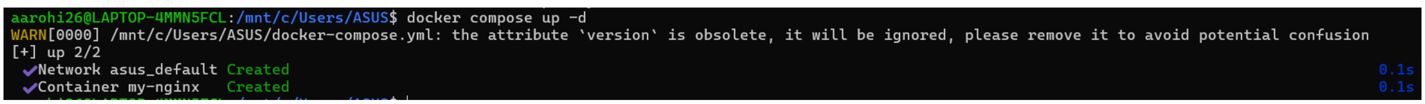
```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker run -d   --name lab-nginx   -p 8081:80   -v $(pwd)/html:/usr/share/nginx/html   nginx:alpine
Unable to find image 'nginx:alpine' locally
alpine: Pulling from library/nginx
612c0c1df4c5: Pull complete
453da7dbc73e: Pull complete
4a8b0b2a5b19: Pull complete
781ff50d2644: Pull complete
583599bb7d38: Pull complete
82736a35d0e7: Pull complete
6a0ac1617861: Pull complete
aee4e54b3865: Pull complete
e8fc446e336c: Download complete
6192e1e6a438: Download complete
Digest: sha256:8aa63af009a39ecd6c28d61da578a5447378c10bb097a069e3a3e0fb42bb6b19
Status: Downloaded newer image for nginx:alpine
2788759c517aee34245512b77f7cb315b15e3ac5cf249cd24e7ccf08d3e182c8
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$
```

Docker Compose (Declarative)

```yaml
version: '3.8'

services:
  nginx:
    image: nginx:alpine
    container_name: lab-nginx
    ports:
      - "8081:80"
    volumes:
      - ./html:/usr/share/nginx/html
```

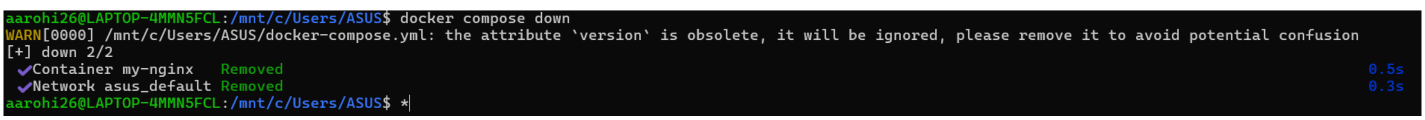
Run:

```
docker compose up -d
```



Stop:

```
docker compose down
```



—

🔷 PART B – PRACTICAL TASKS

Task 1: Single Container

Using Docker Run

```bash
docker run -d \
  --name lab-nginx \
  -p 8081:80 \
  -v $(pwd)/html:/usr/share/nginx/html \
  nginx:alpine
```

```
docker ps
```

```
Run 'docker run --help' for more information
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker ps
CONTAINER ID   IMAGE                      COMMAND                  CREATED          STATUS           PORTS
   NAMES
2788759c517a   nginx:alpine               "/docker-entrypoint.…"   12 minutes ago   Up 12 minutes    0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
   lab-nginx
24f1d004ba3a   containerassgn-mybackend   "uvicorn main:app -- "   2 weeks ago      Up 20 minutes    0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
```

Access:

```
http://localhost:8081
```

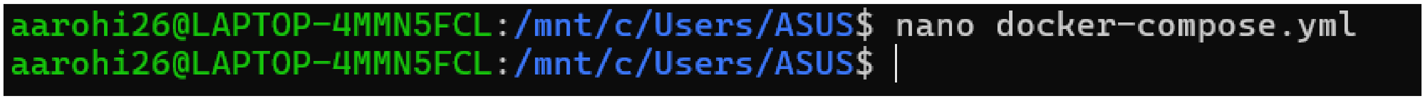
Stop:

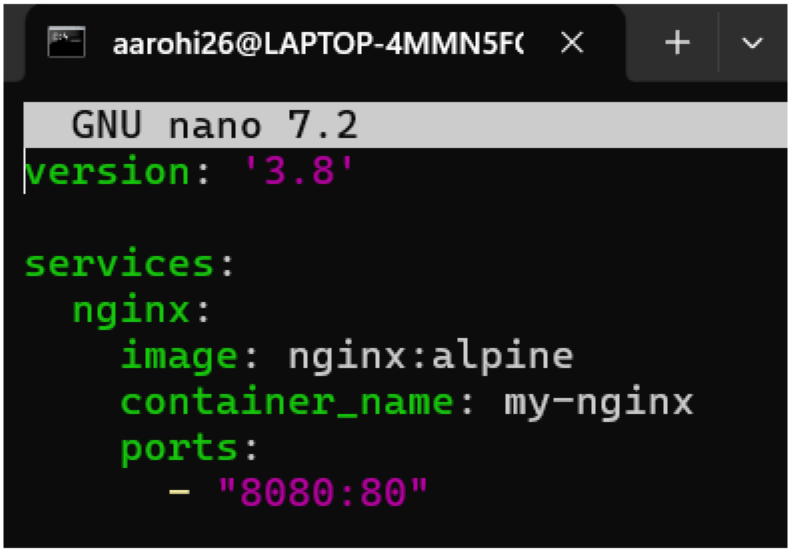
```
docker stop lab-nginx
docker rm lab-nginx
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker stop lab-nginx
docker rm lab-nginx
lab-nginx
lab-nginx
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$
```

Using Docker Compose

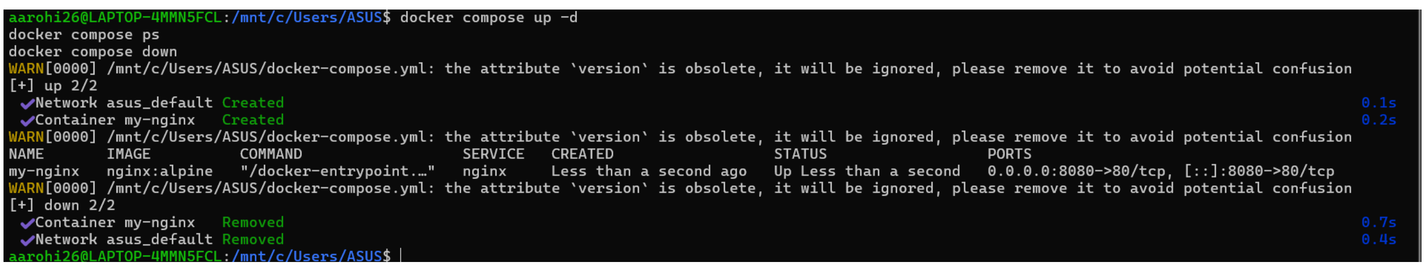
```yaml
version: '3.8'

services:
  nginx:
    image: nginx:alpine
    container_name: lab-nginx
    ports:
      - "8081:80"
    volumes:
      - ./html:/usr/share/nginx/html
```





Run:

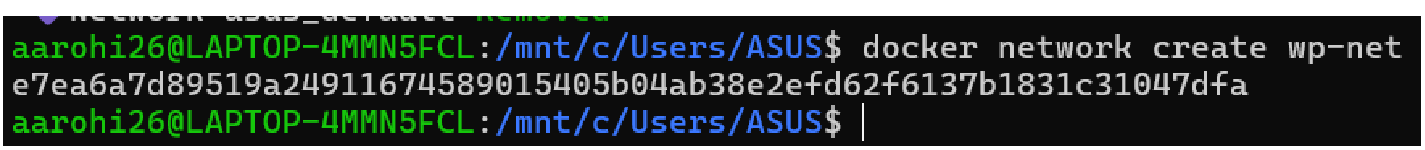
```
docker compose up -d
docker compose ps
docker compose down
```



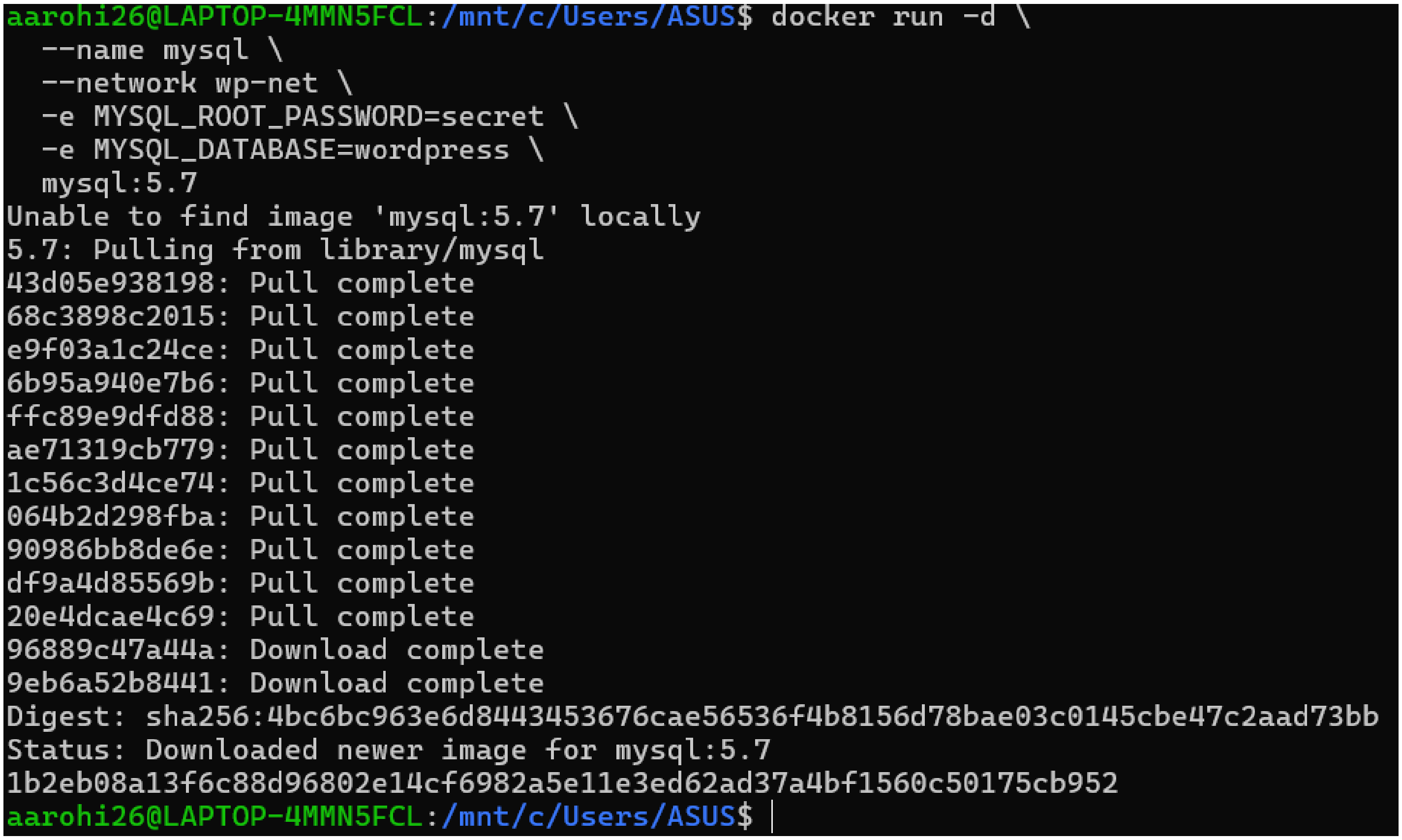
Task 2: Multi-Container (WordPress + MySQL)

Docker Run Approach

```
docker network create wp-net
```



```
docker run -d \
  --name mysql \
  --network wp-net \
  -e MYSQL_ROOT_PASSWORD=secret \
  -e MYSQL_DATABASE=wordpress \
  mysql:5.7
```



```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker run -d \
  --name mysql \
  --network wp-net \
  -e MYSQL_ROOT_PASSWORD=secret \
  -e MYSQL_DATABASE=wordpress \
  mysql:5.7
Unable to find image 'mysql:5.7' locally
5.7: Pulling from library/mysql
43d05e938198: Pull complete
68c3898c2015: Pull complete
e9f03a1c24ce: Pull complete
6b95a940e7b6: Pull complete
ffc89e9dfd88: Pull complete
ae71319cb779: Pull complete
1c56c3d4ce74: Pull complete
064b2d298fba: Pull complete
90986bb8de6e: Pull complete
df9a4d85569b: Pull complete
20e4dcae4c69: Pull complete
96889c47a44a: Download complete
9eb6a52b8441: Download complete
Digest: sha256:4bc6bc963e6d8443453676cae56536f4b8156d78bae03c0145cbe47c2aad73bb
Status: Downloaded newer image for mysql:5.7
1b2eb08a13f6c88d96802e14cf6982a5e11e3ed62ad37a4bf1560c50175cb952
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$
```

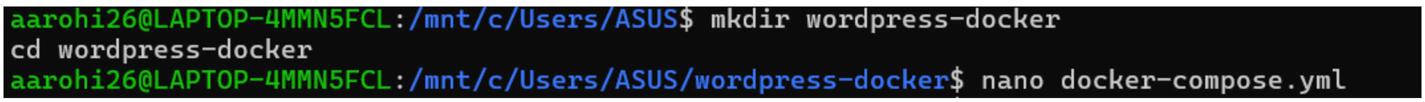
```
docker run -d \
  --name wordpress \
  --network wp-net \
  -p 8082:80 \
  -e WORDPRESS_DB_HOST=mysql \
  -e WORDPRESS_DB_PASSWORD=secret \
  wordpress:latest
```

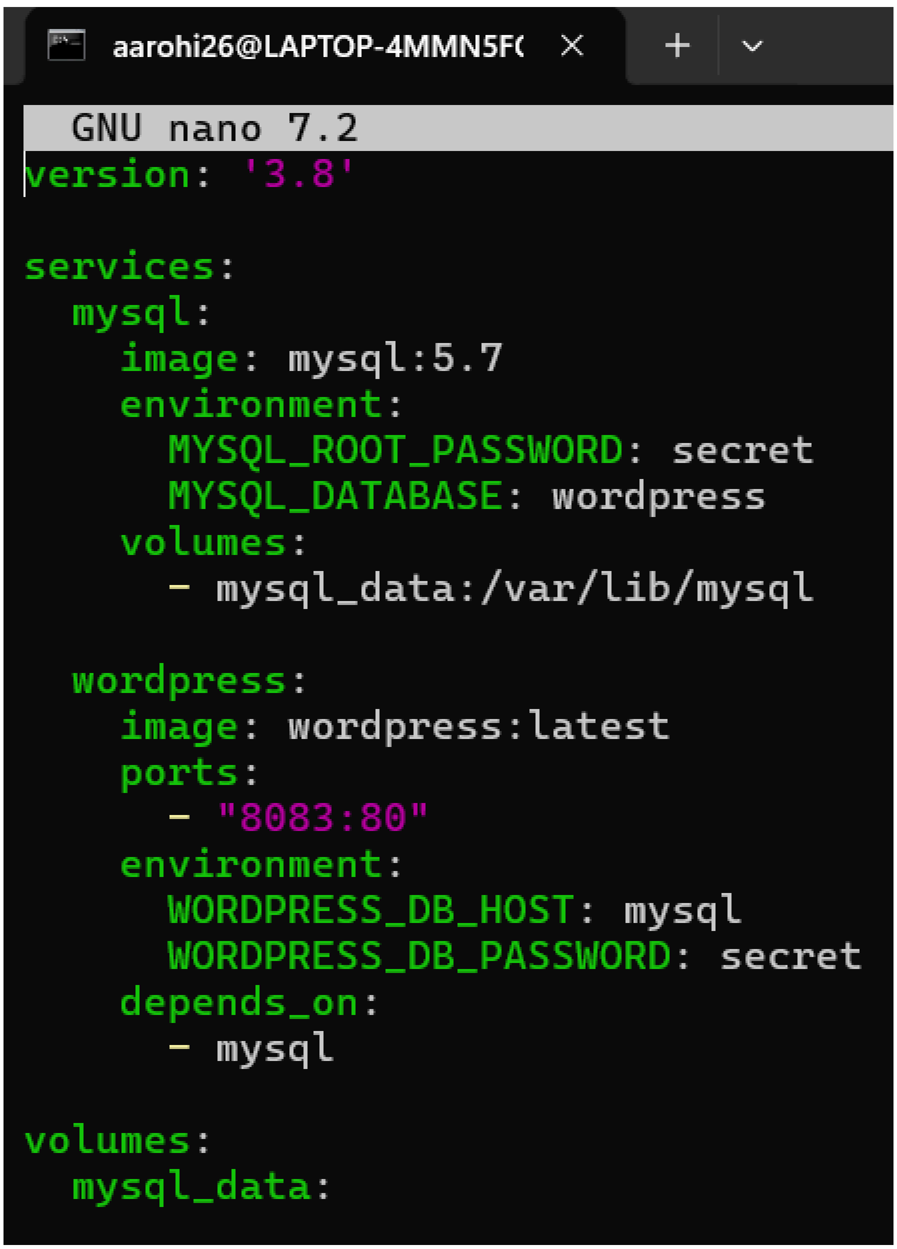
```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker run -d \
  --name wordpress \
  --network wp-net \
  -p 8082:80 \
  -e WORDPRESS_DB_HOST=mysql \
  -e WORDPRESS_DB_PASSWORD=secret \
  wordpress:latest
Unable to find image 'wordpress:latest' locally
latest: Pulling from library/wordpress
78a70974d738: Pull complete
5435b2dcdf5c: Pull complete
0dbbd63ae63a: Pull complete
d5af5dfa43d4: Pull complete
e24121de25e6: Pull complete
4f4fb700ef54: Pull complete
568912d7e28d: Pull complete
3f3cb8cd0960: Pull complete
73ce216c5445: Pull complete
7d9f8a57d57a: Pull complete
de502dcf081a: Pull complete
5020203f7d33: Pull complete
b4b1305df06a: Pull complete
4443d9c71a47: Pull complete
271a7052ce75: Pull complete
9bde0c34d1f4: Pull complete
e052461f1b41: Pull complete
871c6a1a47cc: Pull complete
9cb746f747ab: Pull complete
ef17106e0cde: Pull complete
6562f7228ec3: Pull complete
79e11af4b256: Pull complete
90be0470639d: Pull complete
1fa182c1d458: Pull complete
05bb872ab5e8: Download complete
32a1d2f81bd2: Download complete
Digest: sha256:1a758de2f424f735a9f8ad181783a9ed29e028594869a3323903d92532b35102
Status: Downloaded newer image for wordpress:latest
755b2297cdfee64fab24f58ca0537c5b5db587ea25f053b8ec4dd5d5a888d279
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$
```

Docker Compose Approach

```yaml
version: '3.8'

services:
  mysql:
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: wordpress
    volumes:
      - mysql_data:/var/lib/mysql
```

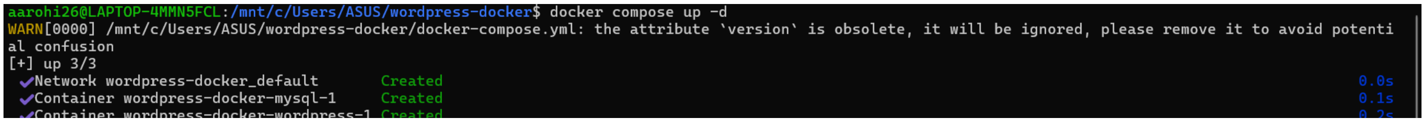
```yaml
  wordpress:
    image: wordpress:latest
    ports:
      - "8082:80"
    environment:
      WORDPRESS_DB_HOST: mysql
      WORDPRESS_DB_PASSWORD: secret
    depends_on:
      - mysql

volumes:
  mysql_data:
```

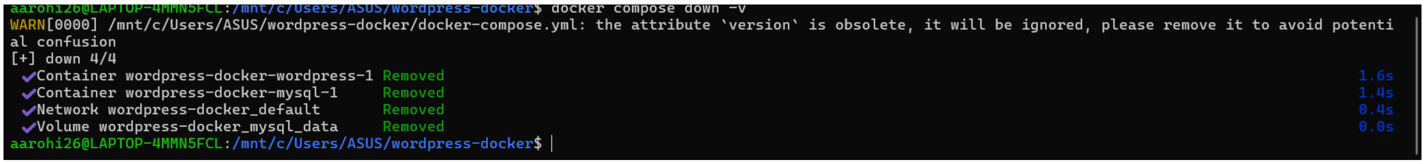




Run:

```
docker compose up -d
docker compose down -v
```





—

🔷 PART C – CONVERSION TASKS

Task 3 – Problem 1 (Web App)

```yaml
version: '3.8'

services:
  webapp:
    image: node:18-alpine
    container_name: webapp
    ports:
      - "5000:5000"
    environment:
      APP_ENV: production
      DEBUG: false
    restart: unless-stopped
```

Task 3 – Problem 2 (Volume + Network)

```yaml
version: '3.8'

services:
  postgres-db:
    image: postgres:15
    container_name: postgres-db
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secret
```

```yaml
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks:
      - app-net

  backend:
    image: python:3.11-slim
    container_name: backend
    ports:
      - "8000:8000"
    environment:
      DB_HOST: postgres-db
      DB_USER: admin
      DB_PASS: secret
    depends_on:
      - postgres-db
    networks:
      - app-net

volumes:
  pgdata:

networks:
  app-net:
```

Run:

```
docker compose up -d
docker compose down -v
```

Task 4 – Resource Limits

```yaml
version: '3.8'

services:
  limited-app:
    image: nginx:alpine
    container_name: limited-app
    ports:
      - "9000:9000"
    restart: always
    deploy:
      resources:
        limits:
```

```
          cpus: "0.5"
          memory: 256M
```

🔍 Explanation

- `deploy` works **only in Docker Swarm**
- In normal Compose → ignored
- Swarm → enforces CPU & memory limits

🔷 PART D – DOCKERFILE WITH COMPOSE

Task 5 – Node App Build

app.js

```
const http = require('http');

http.createServer((req, res) => {
  res.end("Docker Compose Build Lab");
}).listen(3000);
```

Dockerfile

```
FROM node:18-alpine

WORKDIR /app
COPY app.js .

EXPOSE 3000

CMD ["node", "app.js"]
```

docker-compose.yml

```
version: '3.8'

services:
  nodeapp:
```

```yaml
    build:
      context: .
      dockerfile: Dockerfile
    container_name: custom-node-app
    ports:
      - "3000:3000"
```

Run:

```
docker compose up --build -d
```

🔍 image vs build

- `image:` → pulls prebuilt image
- `build:` → creates custom image from Dockerfile

Task 6 – Multi-Stage Dockerfile

Example (Node)

```dockerfile
# Stage 1 - Build
FROM node:18-alpine as builder
WORKDIR /app
COPY . .
RUN npm install

# Stage 2 - Production
FROM node:18-alpine
WORKDIR /app
COPY --from=builder /app .

EXPOSE 3000
CMD ["node", "app.js"]
```

Compose

```yaml
version: '3.8'

services:
```

```yaml
  app:
    build: .
    ports:
      - "3000:3000"
    environment:
      NODE_ENV: production
    volumes:
      - .:/app
```

Check size:

```
docker images
```

🔷 EXPERIMENT 6B – WORDPRESS PROJECT

docker-compose.yml

```yaml
version: '3.9'

services:
  db:
    image: mysql:5.7
    container_name: wordpress_db
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass
    volumes:
      - db_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    container_name: wordpress_app
    depends_on:
      - db
    ports:
      - "8080:80"
    restart: always
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wpuser
```

```yaml
      WORDPRESS_DB_PASSWORD: wppass
      WORDPRESS_DB_NAME: wordpress
    volumes:
      - wp_data:/var/www/html

volumes:
  db_data:
  wp_data:
```

Run:

```
docker compose up -d
docker ps
```

Access:

```
http://localhost:8080
```

Stop:

```
docker compose down
```

🔷 SCALING

```
docker compose up --scale wordpress=3
```

⚠️ Issues:

- Port conflict
- No load balancing

Solution: Add Nginx reverse proxy

🔷 DOCKER SWARM

```
docker swarm init
docker stack deploy -c docker-compose.yml wpstack
docker service scale wpstack_wordpress=3
```

✅ FINAL LEARNING

✔️ Docker Run → quick but messy ✔️ Docker Compose → clean & scalable ✔️ Dockerfile → custom builds ✔️ Multi-stage → optimized images ✔️ Swarm → production scaling

🚀 DONE

You now have:

- Single container setup
- Multi-container app
- Conversion skills
- Build-based deployment
- Scaling knowledge